

[COSE341-01 운영체제 실습#2]

2018320184 강민규

1. write system call 작동 과정 분석

1) C library wrapper 호출 및 parameter 설정

목적: kernel 진입 전, system call 처리에 필요한 data를 준비한다.

data 및 control flow: user program이 write()를 호출하면 C library wrapper가 실행된다. wrapper는 write의 고유 system call 번호와 data 연산에 필요한 parameter들(파일 디스크립터, 버퍼 주소 등)을 규약에 따라 CPU register에 저장한다.

2) syscall 명령어 실행 (권한 모드 전환)

목적: CPU 권한을 격상하여 control flow를 kernel 영역으로 안전하게 넘긴다.

data 및 control flow: register 설정이 끝나면 wrapper 함수가 syscall 명령어를 실행한다. 즉시 소프트웨어 trap이 발생하며 CPU는 kernel mode로 전환되고, 실행 흐름이 kernel의 system call 진입점으로 이동한다.

3) system call table 참조

목적: 요청된 작업에 알맞은 실제 kernel 함수의 메모리 위치를 찾는다.

data 및 control flow: 제어권을 넘겨받은 kernel은 먼저 register에 저장된 system call 번호를 읽어온다. 이 번호를 인덱스로 삼아 배열 구조인 system call table을 참조하여, mapping되어 있는 실제 함수의 주소를 얻는다.

4) kernel handler 함수 실행

목적: 요청받은 실제 물리적 I/O 연산(data 쓰기)을 수행한다.

data 및 control flow: 탐색한 주소로 분기하여 kernel handler 함수를 실행한다. register로 전달받은 user space의 data buffer를 참조하여, 실제 file system이나 장치에 data를 기록하는 control flow를 처리한다.

5) 결과 반환 및 sysret 실행

목적: 작업 결과를 전달하고 CPU 권한을 원래대로 복구한다.

data 및 control flow: handler 함수가 성공적으로 기록한 byte수(or 에러 코드)를 반환 register에 저장한다. 이후 kernel은 sysret 명령어를 통해 user mode로 권한을 되돌린다. 마지막으로 C library wrapper가 이 반환값을 확인하여 최종적으로 application에 전달하

며 전체 과정을 종료한다.

2. system call 생성 및 kernel 컴파일 실습

① system call table에 새로운 system call 등록

460	common	lsm_set_self_attr	sys_lsm_set_self_attr
461	common	lsm_list_modules	sys_lsm_list_modules
462	common	print_student_name	sys_print_student_name
463	common	print_student_id	sys_print_student_id
464	common	print_student_info	sys_print_student_info

462번에 sys_print_student_name을, 463번에 sys_print_student_id를, 464번에 sys_print_student_info system call을 등록했다.

User space의 process가 특정 번호(462 등)를 통해 system call을 호출할 때, kernel이 system call table을 참조하여 해당 번호와 매칭되는 정확한 kernel 함수(handler)의 메모리 주소를 찾아 control flow를 제어할 수 있도록 하기 위한 절차이다.

② syscall.h에 새로운 system call 등록

```
GNU nano 7.2 /usr/src/linux-source-6.8.0/include
/* for __ARCH_WANT_SYS_IPC */
long ksys_semtimedop(int semid, struct sembuf __user *tsops,
                     unsigned int nsops,
                     const struct __kernel_timespec __user *timeout);
long ksys_semget(key_t key, int nsems, int semflg);
long ksys_old_semctl(int semid, int semnum, int cmd, unsigned long arg);
long ksys_msgget(key_t key, int msgflg);
long ksys_old_msgctl(int msqid, int cmd, struct msqid_ds __user *buf);
long ksys_msgrcv(int msqid, struct msgbuf __user *msgp, size_t msgsz,
                 long msgtyp, int msgflg);
long ksys_msgsnd(int msqid, struct msgbuf __user *msgp, size_t msgsz,
                 int msgflg);
long ksys_shmget(key_t key, size_t size, int shmflg);
long ksys_shmdt(char __user *shmaddr);
long ksys_old_shmctl(int shmid, int cmd, struct shmid_ds __user *buf);
long compat_ksys_semtimedop(int semid, struct sembuf __user *tsems,
                             unsigned int nsops,
                             const struct old_timespec32 __user *timeout);
long __do_semtimedop(int semid, struct sembuf *tsems, unsigned int nsops,
                     const struct timespec64 *timeout,
                     struct ipc_namespace *ns);

int __sys_getsockopt(int fd, int level, int optname, char __user *optval,
                     int __user *optlen);
int __sys_setsockopt(int fd, int level, int optname, char __user *optval,
                     int __user *optlen);
asmlinkage long sys_print_student_name(char __user *name);
asmlinkage long sys_print_student_id(char __user *id);
asmlinkage long sys_print_student_info(char __user *school, char __user *major);
#endif
```

```
asmlinkage long sys_print_student_name(char __user *name);
```

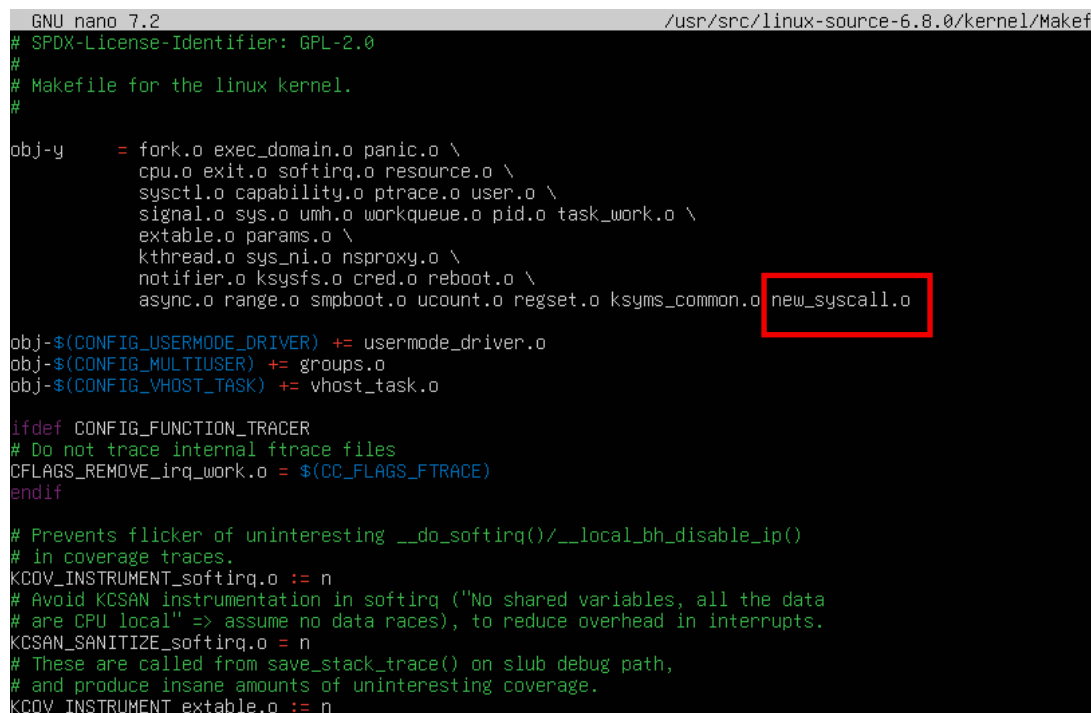
```
asmlinkage long sys_print_student_id(char __user *id);
```

```
asmlinkage long sys_print_student_info(char __user *school, char __user *major);
```

위 세 개의 함수 프로토타입을 추가했다.

이는 kernel 내 다른 source code 파일에서 새로 추가된 system call 함수를 호출하거나 참조할 때, 함수의 형태(반환형 및 parameter type)를 컴파일러에 명시적으로 알려주어 컴파일 단계에서의 타입 오류를 방지하고 정상적인 linking을 보장하기 위함이다.

③ Makefile에 새로운 system call 함수 파일 추가



```
GNU nano 7.2 /usr/src/linux-source-6.8.0/kernel/Makefile
# SPDX-License-Identifier: GPL-2.0
#
# Makefile for the linux kernel.
#
obj-y      = fork.o exec_domain.o panic.o \
             cpu.o exit.o softirq.o resource.o \
             sysctl.o capability.o ptrace.o user.o \
             signal.o sys.o umh.o workqueue.o pid.o task_work.o \
             extable.o params.o \
             kthread.o sys_ni.o nsproxy.o \
             notifier.o ksysfs.o cred.o reboot.o \
             async.o range.o smpboot.o ucount.o regset.o ksyms_common.o new_syscall.o

obj-$(CONFIG_USERMODE_DRIVER) += usermode_driver.o
obj-$(CONFIG_MULTUSER) += groups.o
obj-$(CONFIG_VHOST_TASK) += vhost_task.o

ifdef CONFIG_FUNCTION_TRACER
# Do not trace internal ftrace files
CFLAGS_REMOVE_irq_work.o = $(CC_FLAGS_FTRACE)
endif

# Prevents flicker of uninteresting __do_softirq()/__local_bh_disable_ip()
# in coverage traces.
KCOV_INSTRUMENT_softirq.o := n
# Avoid KCSAN instrumentation in softirq ("No shared variables, all the data
# are CPU local" => assume no data races), to reduce overhead in interrupts.
KCSAN_SANITIZE_softirq.o = n
# These are called from save_stack_trace() on slub debug path,
# and produce insane amounts of uninteresting coverage.
KCOV_INSTRUMENT_extable.o := n
```

kernel directory의 Makefile 내 obj-y 변수 리스트의 끝에 새로 작성한 new_syscall.o를 추가했다.

obj-y는 kernel 빌드 시 기본적으로 컴파일되어 kernel에 포함될 오브젝트 파일 목록이다. 이를 추가함으로써 make 도구가 new_syscall.c source code를 오브젝트 파일(.o)로 컴파일하고, 이를 최종적인 kernel image에 static하게 빌드 및 포함하도록 지시한다.

④ 작성한 new_syscall.c

```
SYSCALL_DEFINE1(print_student_name, char __user *, name) {
    char buf[256];
    long ret;

    ret = strncpy_from_user(buf, name, sizeof(buf));
    if (ret < 0) {
        return -EFAULT;
    }
    if (ret == sizeof(buf)) {
        buf[sizeof(buf) - 1] = '\0';
    }

    printk(KERN_INFO "My Name is %s\n", buf);
    return 0;
}

SYSCALL_DEFINE1(print_student_id, char __user *, id) {
    char buf[256];
    long ret = strncpy_from_user(buf, id, sizeof(buf));

    if (ret < 0) {
        return -EFAULT;
    }
    buf[sizeof(buf) - 1] = '\0';

    printk(KERN_INFO "My Student ID is %s\n", buf);
    return 0;
}

SYSCALL_DEFINE2(print_student_info, char __user *, school, char __user *, major) {
    char school_buf[256];
    char major_buf[256];
    long ret;

    ret = strncpy_from_user(school_buf, school, sizeof(school_buf));
    if (ret < 0) {
        return -EFAULT;
    }
    school_buf[sizeof(school_buf) - 1] = '\0';

    ret = strncpy_from_user(major_buf, major, sizeof(major_buf));
    if (ret < 0) {
        return -EFAULT;
    }
    major_buf[sizeof(major_buf) - 1] = '\0';

    printk(KERN_INFO "I go to %s\n", school_buf);
    printk(KERN_INFO "I major in %s\n", major_buf);
    return 0;
}

%kmq@2018320184:~/workspace$
```

총 3개의 system call 함수로 구성되어 있다. kernel이 user memory space에 직접 접근하는 것은 시스템 안정성 및 보안상 제한되므로, 모든 함수는 공통적으로 `strncpy_from_user` 함수를 사용하여 user space의 문자열 포인터(char __user *) 데이터를 kernel 공간의 내부 buffer로 먼저 안전하게 복사하는 과정을 거친다. 각 함수의 세부 내용은 아래와 같다.

`print_student_name`(단일 인자; DEFINE1 매크로는 인자 1개를 받는다.)

user space로부터 학생 이름의 문자열 포인터(name)를 인자로 전달받는다. kernel buffer(buf)로 복사가 성공적으로 완료되면, `printk` 함수(=kernel 영역에서 C 표준 library 대신 사용하는 kernel 전용 logging 함수로, 출력된 메시지는 kernel ring buffer에 저장되며 user space에서는 `dmesg` 명령어를 통해 이를 조회 가능)를 호출하여 kernel ring buffer에 KERN_INFO 수준으로 "My Name is [이름(Mingyu Kang)]" 형식의 log를 출력한다.

`print_student_id`(단일 인자)

user space로부터 학생 학번의 문자열 포인터(id)를 인자로 전달받는다. 마찬가지로 data 복사 및 검증 과정을 거친 후, "My Student ID is [학번(2018320184)]" 형식으로 kernel log를 출력하도록 구현했다.

print_student_info(다중 인자):

앞선 함수들과 달리 2개의 인자를 받는 SYSCALL_DEFINE2 매크로를 사용하여, 학교(school)와 전공(major) 정보가 담긴 2개의 문자열 포인터를 전달받는다. 전달받은 data를 각각 독립된 kernel buffer(school_buf, major_buf)로 복사한 뒤, "I go to [학교 (Korea University)]" 및 "I major in [전공(Computer Science and Engineering)]" 형식으로 두 줄의 log를 연속하여 출력한다.

⑤ .config 파일 수정

```
#
# Certificates for signature checking
#
CONFIG_MODULE_SIG_KEY="certs/signing_key.pem"
CONFIG_MODULE_SIG_KEY_TYPE_RSA=y
# CONFIG_MODULE_SIG_KEY_TYPE_ECDSA is not set
CONFIG_SYSTEM_TRUSTED_KEYRING=y
CONFIG_SYSTEM_TRUSTED_KEYS=""
CONFIG_SYSTEM_EXTRA_CERTIFICATE=y
CONFIG_SYSTEM_EXTRA_CERTIFICATE_SIZE=4096
CONFIG_SECONDARY_TRUSTED_KEYRING=y
# CONFIG_SECONDARY_TRUSTED_KEYRING_SIGNED_BY_BUILTIN is not set
CONFIG_SYSTEM_BLACKLIST_KEYRING=y
CONFIG_SYSTEM_BLACKLIST_HASH_LIST=""
CONFIG_SYSTEM_REVOCATION_LIST=""
CONFIG_SYSTEM_REVOCATION_KEYS=""
# CONFIG_SYSTEM_BLACKLIST_HASH_LIST is not set
# end of Certificates for signature checking

CONFIG_BINARY_PRINTF=y
```

CONFIG_SYSTEM_TRUSTED_KEYS 및 CONFIG_SYSTEM_REVOCATION_KEYS 항목의 설정 값을 빈 문자열("")로 변경했다. Ubuntu kernel source에는 기본적으로 모듈 서명용 키 경로가 설정되어 있으나, 로컬 빌드 환경에서는 해당 키 파일이 존재하지 않아 X.509 인증서 관련 오류가 발생한다. kernel 빌드 시 발생할 수 있는 이러한 인증 오류를 무시하고 컴파일을 정상적으로 완료하기 위해 해당 설정을 비활성화한 것이다.

⑥ kernel 컴파일 후 버전 확인

```
kmg@2018320184:~/workspace$ uname -r
6.8.12
kmg@2018320184:~/workspace$
```

uname -r 명령어는 현재 부팅되어 동작 중인 kernel의 release version을 출력한다. 그 결과가 기존 Ubuntu의 기본 kernel version(예: 6.8.0-xxx)이 아닌 실습을 통해 직접 빌드한 6.8.12로 확인됨으로써, system이 새로 컴파일된 user defined kernel로 정상적으로 교체 및 부팅되었음을 객관적으로 증명한다.

⑦ 작성한 call_new_syscall.c

```
kmg@2018320184:~/workspace$ uname -r
6.8.12
kmg@2018320184:~/workspace$ cat ~/workspace/call_new_syscall.c
#include <unistd.h>

int main(void) {
    char name[] = "Mingyu Kang";
    char id[] = "2018320184";
    char school[] = "Korea University";
    char major[] = "Computer Science and Engineering";
    syscall(462, name);
    syscall(463, id);
    syscall(464, school, major);
    return 0;
}
```

C 표준 library(unistd.h)에서 제공하는 syscall() 함수를 syscall(462, name)과 같이 첫 번째 parameter'로 system call table에 등록한 고유 system call 번호(462, 463, 464)를 입력하고, 두 번째 parameter'부터는 user space에서 정의한 문자열 변수(name, id 등)의 포인터를 전달하여 소프트웨어 interrupt를 발생시키고(trap) kernel 모드로 진입하도록 코드를 작성했다.

⑧ call_new_syscall.c 컴파일 및 실행 후, dmesg를 사용한 커널 메시지 출력

```
[ 15.851891] loop0: detected capacity change from 0 to 8
[ 24.405550] systemd-journal[347]: /var/log/journal/2bbc78c90e6c486b9b04b69aa06c255d/user-1000.journal: Journal file uses a
ating.
[ 68.407592] workqueue: e1000_watchdog [e1000] hogged CPU for >10000us 4 times, consider switching to WQ_UNBOUND
[ 71.886703] workqueue: blk_mq_run_work_fn hogged CPU for >10000us 4 times, consider switching to WQ_UNBOUND
[ 79.486418] workqueue: e1000_watchdog [e1000] hogged CPU for >10000us 8 times, consider switching to WQ_UNBOUND
[ 82.836576] workqueue: drm_fb_helper_damage_work hogged CPU for >10000us 8 times, consider switching to WQ_UNBOUND
[ 104.563345] workqueue: e1000_watchdog [e1000] hogged CPU for >10000us 16 times, consider switching to WQ_UNBOUND
[ 104.563345] watchdog check: cs_nsec: 5000502588 wd_nsec: 5000500042
[ 413.459654] My Name is Mingyu Kang
[ 413.459694] My Student ID is 2018320184
[ 413.459695] I go to Korea University
[ 413.459695] I major in Computer Science and Engineering
```

call_new_syscall.c 프로그램은 user space에 data를 출력하는 printf 구문이 없으므로 terminal에는 아무 메시지가 나타나지 않는다. 그러나 kernel 메시지를 확인하는 dmesg 명령어를 통해 출력된 log를 확인하면, 작성했던 new_syscall.c 내부의 printk 구문에 의해 학생 이름, 학번 등의 문자열이 kernel ring buffer에 정상적으로 기록된 것을 확인할 수 있다. 이는 system call 호출, parameter 전달, kernel 내부 함수 실행이 모두 에러 없이 성공적으로 수행되었음을 의미한다.